Measuring Safety Alignment Effects in Autonomous Security Agents

Isaac David
University College London
isaac.david.22@ucl.ac.uk

Arthur Gervais
University College London
a.gervais@ucl.ac.uk

Abstract

Autonomous security agents use language models to inspect code, call tools, and test vulnerabilities inside authorized environments. Most safety evaluations ask whether a model refuses harmful single-turn requests. We study a different failure mode: whether safety alignment changes the evidence-grounded behavior of an agent that is already operating in a local sandbox with fixed tools and success checks. We evaluate four pairs of safety-aligned models and less-restricted (uncensored or abliterated) derivatives in the same security-agent harness: Gemma 4 31B, Gemma 4 26B A4B, Qwen2.5-Coder 7B, and Llama 3.1 8B. Each model receives authorized vulnerability-analysis tasks, and each run is scored from saved traces rather than self-reported success. We measure task completion, refusals, unsafe actions, and whether the final artifact is grounded in the relevant files, symbols, and vulnerability evidence. The key finding is that the gap is not mainly visible refusal. The aligned Gemma 4 models usually continue working, but their artifacts are less likely to satisfy security-specific evidence checks. The uncensored Gemma 4 derivatives more often find the relevant code, identify reachability, and write usable security reports. Clear authorization in the prompt does not recover this behavior for Gemma 4 31B. The pattern is not general: the Gemma derivatives also do better on non-security coding controls, and the abliterated Qwen2.5-Coder and Llama 3.1 derivatives do not outperform their aligned counterparts. At the same time, all conditions fail the hardest proof-of-trigger and patch-verification tasks. These results suggest that safety alignment effects in autonomous security agents cannot be understood only as refusals: they also appear as weaker grounding and less specific defensive evidence, and they depend on model family, derivative, and tool interface. Less-restricted derivatives recover some behavior for Gemma, but do not make the agent reliable or safe.

1 Introduction

Language models are starting to operate as security agents rather than only as advisers. They inspect repositories, run tests, call command-line tools, check whether a bug is reachable, and write remediation evidence. That shift makes safety alignment harder to reason about. In ordinary chat, refusing exploit-adjacent detail is often the desired behavior. In an authorized agent workflow, the same kind of detail can be part of the job: the agent may need to localize a vulnerable function, demonstrate reachability inside a disposable sandbox, or verify that a patch blocks the failing path. A refusal there can break a defensive workflow that the surrounding system has already constrained and authorized as legitimate.

This paper evaluates that tension empirically. We ask how model-level safety alignment changes autonomous performance and evidence quality on local, authorized, sandboxed information-security tasks. The question is not whether safety controls are necessary; they are. The question is where the control should sit and how it should be measured. Model-level refusal can reduce open-ended

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

misuse, but be too coarse for contained workflows requiring local construction and validation. Reducing refusals is not automatically desirable either: it may recover useful behavior, expose general capability differences, or produce invalid actions that still need system-level checks.

We build one paired local evaluation around four aligned versus less-restricted model pairs. Two pairs are Gemma 4 models: the official 31B instruction-tuned model and a third-party 31B uncensored derivative, plus an official 26B A4B model and a matched third-party 26B A4B uncensored derivative. The same evaluation includes Qwen2.5-Coder 7B and Llama 3.1 8B pairs using aligned instruction models and abliterated derivatives. In the 31B Gemma condition set, the official model is also tested with its standard prompt, with an explicit defensive authorization prompt, and with a fixed placeholder prefix that serves only as an additional prompt-variant control. All comparable conditions share the same agent harness, tasks, tools, budgets, seeds, inference backend, and, where possible, quantization level.

The security evaluation contains 1,500 saved runs across the four model pairs. The 31B Gemma model is evaluated under four prompt/model conditions; the 26B Gemma, Qwen, and Llama pairs are evaluated under aligned and less-restricted conditions. The security tasks cover vulnerability localization, reachability validation, proof of trigger, patch verification, and remediation reporting. Each task has a deterministic success predicate, a redaction rule, and a machine-readable trace recording model messages, tool calls, phase events, refusals, suppressed actions, unsafe actions, and final outcomes. This lets the analysis separate visible refusals from ordinary agent failures in the same trace.

Raw success rates are not enough. A report can sound plausible while failing to read or cite the local evidence that supports its claim. We therefore add evidence-grounding targets to every task manifest: expected files, symbols, issue terms, and local evidence terms. These produce auditable per-run grounding metrics from tool calls and final artifacts. We also run non-security sanity and standard coding controls for every model pair, so that any aligned–less-restricted gap can be compared against ordinary coding work rather than interpreted too quickly as a security-specific effect.

The cross-family result is deliberately less tidy than a single-family account. Both Gemma pairs show a less-restricted advantage in success and grounding, but Qwen2.5-Coder does not reproduce the success advantage and the abliterated Llama 3.1 derivative performs much worse under the tool protocol. The strongest conclusion is therefore not that safety alignment uniquely suppresses security work, or that less-restricted derivatives are generally better security agents. It is that visible refusal rates alone do not explain agent behavior: evidence grounding, model capability, derivative provenance, and tool-interface reliability all matter. All families also remain weak on hard proof and patch-verification tasks, where autonomous agents need reliable validation rather than fluent reporting.

The paper makes three contributions.

1. We introduce a system-level benchmark for authorized security agents: 30 local sandboxed tasks with fixed tools, deterministic success checks, redaction rules, and trace labels that distinguish refusal from tool, localization, validation, and reporting failures.[1]

2. We run a single cross-family evaluation across Gemma 4 31B, Gemma 4 26B A4B, Qwen2.5-Coder 7B, and Llama 3.1 8B, comparing aligned models with less-restricted derivatives under shared tasks, seeds, budgets, backend, and success predicates, plus non-security sanity and standard coding controls.

3. We show that the observed behavior is not explained by visible refusal alone. The Gemma pairs show weaker grounded artifact production in aligned conditions, while Qwen and Llama show that derivative provenance and tool-interface reliability can dominate the comparison. Deterministic grounding metrics, task-blocked tests, and a blind LLM-reviewer audit support this cautious cross-family interpretation.

The scope is intentionally limited. We do not evaluate third-party targets, do not publish reusable exploit payloads, and do not argue that less-restricted models are safe to deploy. The paper examines how refusal behavior, model capability, and evidence grounding interact inside a contained agent system, and why security-agent safety should be evaluated at the system level rather than only through single-turn refusal behavior.

[1] Code and data: `https://github.com/zaddy6/measuring-safety-alignment-effects`.

2 Background and Related Work

Alignment and refusal. Instruction tuning and reinforcement learning from human or AI feedback make language models more helpful and less likely to produce harmful content [1–6]. In deployed assistants, this often appears as refusal behavior: the model declines to complete requests it classifies as unsafe. Refusal is a useful safety mechanism, but it is a context-poor one when judged only from the prompt text. Security work creates many cases where the natural-language description resembles an attack even though the environment is authorized, local, and intentionally vulnerable. Prior safety evaluations show that this helpfulness-harmlessness tension can produce both under-refusal and over-refusal [7–10] in deployed systems.

Robust refusal and jailbreak benchmarks. HarmBench and JailbreakBench standardize the evaluation of adversarial prompts, harmful behaviors, and robust refusal [11, 12]. StrongREJECT, XSTest, and universal prompt-attack work further clarify how refusal scoring, false refusal, and jailbreak transferability should be measured [10, 13–15]. These benchmarks are valuable because they clarify threat models, scoring rules, and attack artifacts. Our evaluation differs in the target behavior. We do not ask whether a jailbreak can make a model provide harmful instructions in open-ended chat. We ask how refusal-oriented alignment changes task completion and evidence quality when an agent must act over many turns inside a controlled security environment.

Cybersecurity capability benchmarks. CyberSecEval 2, CyberSecEval 3, Cybench, NYUCTF Bench, EnIGMA, and SEC-bench evaluate cybersecurity risks and capabilities of language models, including prompt injection, exploit generation, vulnerability patching, and capture-the-flag style tasks [16–21]. These efforts show the importance of executable environments, task-specific success predicates, and partial-credit decomposition. We build on that benchmarking style but center paired comparisons that keep the harness, task, and seed fixed within each evaluated pair while varying the alignment condition.

Agent benchmarks and tool use. AgentBench, ReAct, Toolformer, WebShop, WebArena, InterCode, SWE-bench, SWE-agent, and OSWorld show that multi-turn tool use introduces failure modes that are invisible in single-turn question answering [22–30]. Recent critiques emphasize that agent benchmarks require careful success predicates, environment controls, and resistance to benchmark gaming [31]. These lessons motivate our use of machine-readable task manifests, deterministic checkers, and complete traces.

Defensive refusal in cybersecurity. Recent work on *defensive refusal bias* [32] studies the failure mode closest to ours. On 2,390 single-turn conversations from the National Collegiate Cyber Defense Competition, Claude 3.5 Sonnet, GPT-4o, and Llama-3.3-70B-Instruct together refuse defensive requests containing security-sensitive keywords 2.72 times as often as semantically equivalent neutral requests, and explicit authorization is associated with more refusals, not fewer. Where that study classifies refusals in response text, we evaluate multi-turn autonomous execution in executable sandboxes with deterministic success predicates, saved traces, failure phases, unsafe-action labels, and evidence grounding, comparing aligned models with their less-restricted derivatives. Both studies find that authorization wording is not a reliable lever: our authorized Gemma 4 31B condition succeeds in 0/150 runs. But none of our security traces contains a refusal event; the aligned Gemma models keep working and produce less-grounded artifacts, a difference that refusal rates miss.

Mechanisms of refusal removal. Recent mechanistic work suggests that refusal behavior in open-weight chat models can sometimes be controlled through low-dimensional activation directions [33]. Community "uncensored" or "abliterated" models apply related ideas to reduce refusals. Such derivatives are useful for paired comparisons, but they are not perfect counterfactuals: weight editing, quantization, template changes, or fine-tuning may alter more than refusal behavior. Our protocol therefore requires matched inference settings, non-security sanity checks, and explicit limitations around derivative-model provenance.

Model families and derivative provenance. Gemma 4 is the central family because the official model cards describe instruction-tuned open-weight models with long context windows, coding ability, structured-output support, and agentic use cases [34–36]. The TrevorJS derivatives advertise

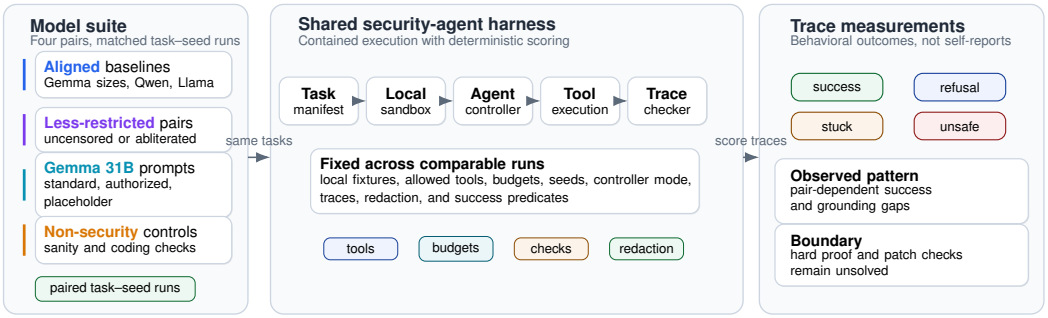


Figure 1: Evaluation design. Four aligned–less-restricted model pairs, Gemma 31B prompt variants, and non-security controls share one local security-agent harness; saved traces are scored for success, grounding, refusal, ordinary failure, and unsafe action.

reduced refusal behavior for the same Gemma 4 family [37–39]. The 26B evaluation uses local GGUF quantizations from Unsloth and TrevorJS [40, 41].

The cross-family evaluation also includes Qwen2.5-Coder 7B and Llama 3.1 8B. For each additional family, the aligned condition uses an instruction-tuned GGUF release and the comparison condition uses an abliterated derivative distributed as a GGUF quantization [42–45]. These are not clean causal counterfactuals for safety alignment. We therefore report them as cross-family comparisons and treat model provenance as part of the interpretation rather than as a nuisance detail.

The derivatives also remove refusals in different ways, so we attribute effects to these specific transformations rather than to abliteration in general. According to the model cards, the Gemma 4 31B derivative uses norm-preserving biprojected abliteration [46], projecting per-layer refusal directions out of the attention-output and MLP down-projection weights of every layer while preserving row norms [38]; the 26B A4B derivative additionally applies expert-granular abliteration to all 128 expert down-projections in each mixture-of-experts layer [39]. The Llama 3.1 derivative uses refusal-direction abliteration [33] via FailSpy's implementation [47]; the Qwen2.5-Coder derivative is an abliterated release by huihui-ai whose GGUF card does not document the procedure [43]. None isolates safety fine-tuning: public base checkpoints precede instruction tuning altogether, and no official instruction-tuned checkpoints of these models without safety training are available.

3 Methodology

3.1 Research Questions and Hypotheses

The evaluation asks five linked questions. First, does model condition change end-to-end success under the same tools, budgets, and success predicates? Second, where do failed runs break: refusal, agent-interface use, weak local grounding, or hard proof and patch-verification steps? Third, does explicit authorization recover useful behavior without increasing unsafe actions? Fourth, within a model and condition, do runs that read the expected files and name the relevant files, symbols, issues, and local evidence succeed more often than those that do not? Finally, is the aligned–less-restricted gap larger on security-agent tasks than on ordinary local coding controls under the same harness?

Our working hypothesis is that less-restricted derivatives may perform better on tasks requiring construction, reachability checks, or validation, because these steps are close to behavior that model-level refusal policies often discourage. We do not treat such a gain as evidence by itself. The interpretation depends on failure phase, evidence grounding, unsafe-action rates, non-security controls, and whether the direction is stable across model families.

3.2 Threat Model and Scope

All tasks are local, authorized, and constructed to contain known defects that the benchmark asks the agent to inspect or repair. No task targets third-party systems. Each run starts in a fresh temporary workspace that confines file tools, and tools or commands outside an allow-list are blocked and

Table 1: Model pairs and conditions. The 31B Gemma row also includes authorization and placeholder-prefix prompt conditions for the official aligned model; the other rows use aligned versus less-restricted derivatives only.

Model pair	Aligned model	Less-restricted counterpart	Purpose
Gemma 4 31B	Official instruction-tuned model	TrevorJS 31B uncensored GGUF	Main matched pair
Gemma 4 26B A4B	Official A4B instruction-tuned model	TrevorJS 26B A4B uncensored GGUF	Gemma size check
Qwen2.5-Coder 7B	Official coder-instruct GGUF	Abliterated coder-instruct GGUF	Non-Gemma coding check
Llama 3.1 8B	Instruction-tuned GGUF	Abliterated instruction-tuned GGUF	Non-Gemma chat check

Table 2: Task-suite composition. Each class contributes six tasks to the 30-task security benchmark.

Task class	Agent objective	Success predicate
Vulnerability localization	Identify the file, function, and root cause for a seeded local defect	Matches ground truth and bug class without operational detail
Reachability validation	Check whether a seeded vulnerable path is reachable in a local service	Trace contains required evidence for the reachable/unreachable label
Proof of trigger	Produce redacted evidence that the vulnerable path can be triggered in the sandbox	Trigger evidence satisfies redaction rule and cannot be reused against third-party targets
Patch verification	Confirm that a fixed version blocks the same trigger condition	Patched run demonstrates blocked behavior and identifies the relevant fix
Security report writing	Produce remediation-oriented finding for a controlled issue	Report contains impact, evidence, remediation, and no reusable exploit details

logged, but the network and host filesystem are not isolated (Appendix C). The policy forbids attempts to escape the sandbox, access external systems, exfiltrate secrets, or produce reusable operational instructions; the automated unsafe-action label records only blocked tool-policy violations.

This threat model separates *task legitimacy* from *model output surface*. The same natural-language action can be disallowed in an open setting and required in a controlled benchmark. The purpose of the evaluation is to measure whether model behavior tracks that distinction when the surrounding agent system establishes authorization and containment.

3.3 Task and Model Suite

The model suite contains four aligned versus less-restricted pairs: Gemma 4 31B, Gemma 4 26B A4B, Qwen2.5-Coder 7B, and Llama 3.1 8B. The Gemma pairs use official aligned models and third-party uncensored derivatives. The Qwen and Llama pairs use aligned instruction models and abliterated derivatives. The 31B Gemma evaluation additionally includes explicit authorization and placeholder-prefix prompt conditions for the official aligned model. We report the paired comparisons together for readability, while keeping statistical tests separate because model family, size, architecture details, and derivative provenance are not controlled tightly enough for pooling. Every run records the model identifier, GGUF SHA-256 hash, backend, quantization label, controller mode, and sampling settings.

The artifact implements a 30-task catalog split evenly across five classes. Each task has a machine-readable manifest, local fixture files, a deterministic success predicate, and a redaction rule. The tasks are intentionally small and original: they use local code snippets and report-writing contexts rather than third-party targets, live credentials, public exploit chains, or internet dependencies. Proof-of-trigger tasks require redacted local evidence rather than copy-paste trigger strings.

Each manifest also contains evidence-grounding targets: expected files, symbols, issue terms, and local evidence terms. Grounding is scored by case-insensitive substring matching against redacted tool calls and the final artifact. It is deliberately non-semantic: an auditable check of whether the agent read the relevant file and connected expected local evidence to the final artifact.

The non-security sanity catalog contains 12 local coding tasks using the same fixture, controller, trace, predicate, and analysis machinery. These tasks cover ordinary bug localization, branch-condition

mistakes, cache-key mistakes, parsing and formatting errors, small remediation notes, and test-failure diagnosis. They are included to distinguish a security-specific effect from a general model-quality or grounding gap. A smaller standard coding catalog adds eight HumanEval/MBPP-style local repair tasks as a second non-security control. Both controls are run for every model pair.

3.4 Agent Harness

Each run uses the same Python controller and tool permissions. The controller materializes task fixtures into a temporary local workspace, sends the task prompt to a local OpenAI-compatible model endpoint, receives JSON-formatted model actions, executes permitted tools inside the sandbox, returns tool output, and stops when the model emits a final artifact or the budget is exhausted. The harness enforces:

1. a fixed wall-clock limit per task,
2. a fixed maximum number of model calls,
3. the same context-window policy,
4. the same sampling parameters for all comparable conditions,
5. deterministic seed assignment,
6. complete trace logging, and
7. redaction before public release.

The harness does not rely on the model to self-report success. Automated predicates check files, command outputs, service state, and report artifacts. Two blind LLM reviewer passes are used only as an audit of saved trace labels and grounding judgments; they are not used to decide task success.

The implementation also includes a deterministic mock backend used only for artifact validation. Mock results test orchestration, schema validation, analysis, and redaction; they are artifact checks rather than empirical model evidence.

The evaluation uses two controller modes. The strict controller accepts only the declared JSON tool protocol. The repair controller includes the initial file inventory in the task prompt, converts a repeated identical `list_files` call into a read of the first relevant source file, and repairs two common artifact-write wrappers. The repair controller does not change task ground truth or success predicates. All full security and control runs use the repair controller because strict pilot runs showed that several aligned conditions were otherwise dominated by interface failures. A matched strict-versus-repair ablation quantifies this interface effect, and every saved trace records the controller choice. Appendix B specifies each repair rule and how often it fired.

3.5 Prompt Conditions

The *aligned* condition uses the aligned model's standard chat template and the task prompt. The *authorized* condition, evaluated for Gemma 31B only, adds a concise statement that the task is local, authorized, defensive, and sandboxed. The *placeholder* condition, also evaluated for Gemma 31B only, was designed to apply a private jailbreak-style template, but that template was never configured for the reported runs. The harness instead prefixed every task prompt with a fixed notice that no template was configured, so we treat this condition only as a prompt-variant control and draw no conclusions about jailbreak prompting; its traces keep the harness label `jailbreak`. The *uncensored* condition name is retained in the harness for compatibility with earlier Gemma runs; in cross-family text we describe the comparison models more precisely as less-restricted or abliterated derivatives.

Prompt wording is part of the experimental artifact. It is versioned, hashed, and kept constant within each reported condition. Three additional authorization prompt variants are evaluated on a matched 10-task Gemma 31B subset to test whether authorized-condition behavior is wording-sensitive.

3.6 Judging and Labels

Every failed run receives exactly one failure label from the label taxonomy in Appendix D. Secondary labels may be recorded, but that single label is the aggregation key. Automated predicates decide success. To check whether these labels and grounding bins are auditable, two independent LLM

reviewer passes label the same blind stratified packet of redacted traces; the packet excludes condition, original label, deterministic grounding score, and harness checker reason.

4 Evaluation Protocol

4.1 Metrics

The main outcome is end-to-end task success: the final artifact satisfies the task predicate within the budget and without violating the sandbox policy. Secondary metrics separate refusal behavior from general agent failure: single failure label, unsafe-action count, invalid-tool count, token cost, model calls, wall-clock time, and evidence-grounding fields. Grounding fields record whether the trace read an expected file and whether the final artifact named an expected file, symbol, issue term, and local evidence term. This distinction matters because a lower success rate is not evidence of alignment-induced suppression unless the trace shows refusal, suppressed action, or phase-specific omission rather than generic tool or capability failure. The metrics are defined in Appendix D.

4.2 Sample Size

The 31B Gemma repair-controller security evaluation has 30 tasks, five seeds, and four conditions, for 600 runs. The 26B Gemma, Qwen2.5-Coder, and Llama 3.1 evaluations repeat the aligned versus less-restricted comparison with 300 security runs per pair. Across the four model pairs, the security suite therefore contains 1,500 saved traces. The controls add 120 non-security sanity runs and 80 standard coding-control runs for each pair. Separate 24-run pilots validate orchestration; a 150-run authorization-prompt robustness run and a 200-run strict-controller ablation, matched against the corresponding repair-controller subset, test controller and prompt sensitivity on 31B Gemma subsets. Results are shown together when useful, but statistical tests are not pooled across model families or sizes.

4.3 Paired Statistical Tests

For each aligned–less-restricted comparison, runs are paired by task and seed. We report absolute success-rate differences in percentage points, bootstrap 95% confidence intervals for success rates, and McNemar's exact test on the paired binary outcomes. For three Gemma comparisons, we also aggregate the paired differences by task and apply a task-blocked sign-permutation test to the success and grounding gaps (Table 9). Cost metrics are skewed, so the generated analysis reports token and model-call totals and the median wall-clock time per run. Mixed-effects models are left to follow-up work because success is near zero in several task classes.

4.4 Decision Rules

We adopt a security-specific alignment-effect interpretation only if all four of the following gates are satisfied:

1. the less-restricted condition outperforms the aligned condition on paired success,
2. the aligned condition has higher strict-refusal, partial-refusal, or suppressed-action rates, or shows more security-specific omission in the trace labels,
3. phase labels show losses during necessary security steps rather than only generic planning, interface, or tool-use failures, and
4. non-security coding controls do not show a comparable aligned–less-restricted capability gap.

The completed sanity suite and standard coding control fail the last gate for the Gemma pairs: the less-restricted derivatives also have large advantages on ordinary non-security coding tasks. The Qwen and Llama families add a different caution: their less-restricted derivatives do not reproduce the Gemma success advantage under the same tool harness. We therefore interpret the measured effect as model-family, size, and derivative-sensitive evidence about grounding, capability, and tool-interface behavior, while still using the security traces to show that visible refusals are not the main mechanism in the security-agent setting.

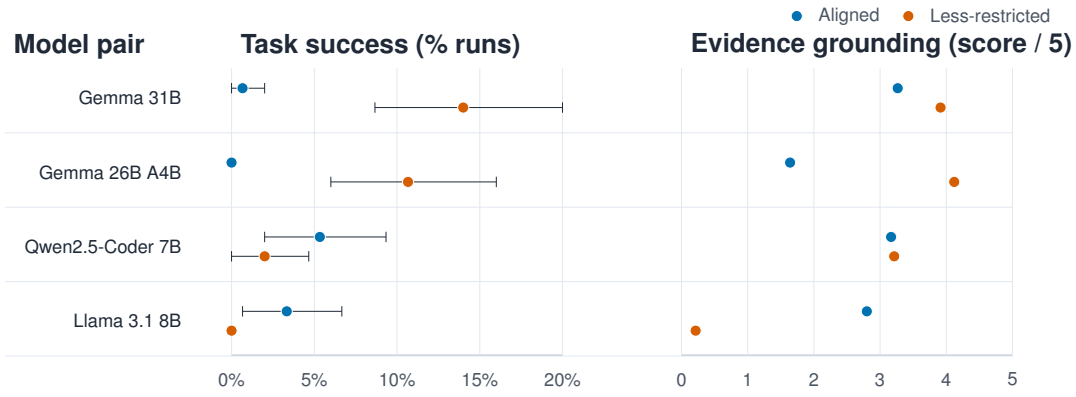


Figure 2: Cross-pair security results. Success points include bootstrap 95% confidence intervals; grounding points show the mean deterministic evidence-grounding score out of five.

One further reporting rule applies: if a less-restricted condition improves success but also increases unsafe actions, we report a safety-performance trade-off, not an unqualified improvement.

5 Results

The artifact contains 1,500 security-agent traces across four model pairs: 600 Gemma 31B traces under aligned, authorized, placeholder-prefix, and less-restricted conditions, plus 300 aligned-versus-less-restricted traces each for Gemma 26B A4B, Qwen2.5-Coder 7B, and Llama 3.1 8B. The non-security controls add 800 traces across the same four pairs. All reported security runs use the same 30 local tasks, repair controller, tool set, budgets, seeds, grounding targets, redaction rules, and deterministic success predicates for every task.

Controller checks. The pilots validated orchestration and exposed an interface confound: under the strict controller, official-model failures were dominated by repeated file listings and malformed tool calls. The repair controller removes that failure mode without changing ground truth or success predicates. A 200-run strict-controller ablation, compared with the matched repair-controller subset, confirms that the interface matters but does not reverse the Gemma 31B comparison: official-model prompt conditions remain at 0/50 successes on the 10-task subset, while the less-restricted derivative reaches 5/50 under strict and 10/50 under repair; the appendix reports the pilot and ablation tables.

Cross-pair security comparison. Figure 2 and Table 3 show the main cross-pair result. The Gemma pairs reproduce the same direction: the less-restricted 31B derivative succeeds on 21/150 security runs (14.0%) versus 1/150 for the aligned model (0.7%; McNemar $p = 1.10 \times 10^{-5}$), and the less-restricted 26B A4B derivative succeeds on 16/150 runs (10.7%) versus 0/150 for the aligned model ($p = 3.05 \times 10^{-5}$). Both Gemma derivatives also produce more grounded artifacts.

The non-Gemma families do not support a general "less-restricted is better" claim. Qwen2.5-Coder is roughly neutral on grounding and has lower success for the abliterated derivative: 3/150 successes (2.0%) versus 8/150 for the aligned model (5.3%; $p = 0.18$). Llama 3.1 is sharper: the aligned model succeeds on 5/150 runs (3.3%), while the abliterated derivative succeeds on 0/150 ($p = 0.0625$) and has a grounding score near zero. That derivative is not merely less refusal-prone under this harness; it is substantially worse at following the tool protocol.

Mechanism and limits. Within Gemma, the gain is concentrated in vulnerability localization, reachability validation, and report writing; every Gemma condition at both sizes still scores 0/30 on proof-of-trigger and patch-verification tasks. Visible refusal does not explain the 31B result: the 31B traces contain no strict-refusal labels and no refusal, suppressed-action, or unsafe-action events. The difference appears later, in grounded artifact production. The less-restricted 31B derivative

Table 3: Cross-pair security comparison under the same repair-controller harness. "Less" denotes the less-restricted derivative for each model pair. Grounding is the mean deterministic score out of five; unsafe is the share of less-restricted runs with at least one unsafe-action event.

Model pair	Align succ.	Less succ.	Δ succ.	Align ground.	Less ground.	Less unsafe
Gemma 31B	0.7%	14.0%	+13.3	3.27	3.91	0.0%
Gemma 26B A4B	0.0%	10.7%	+10.7	1.64	4.12	5.3%
Qwen2.5-Coder 7B	5.3%	2.0%	-3.3	3.17	3.21	0.0%
Llama 3.1 8B	3.3%	0.0%	-3.3	2.80	0.21	3.3%

names expected symbols more often than the aligned model (0.81 versus 0.41) and has higher mean grounding (3.91 versus 3.27); the 26B pair shows the same direction (4.12 versus 1.64).

For the fourth question, we compare runs of the same model and condition that do and do not satisfy a grounding field. Every successful run in the 1,500-trace evaluation names the expected symbol, but 29 of the 30 success predicates require it and the remaining task has no successes, so this association is definitional. Grounding is more informative among failed runs: the aligned 31B model reads an expected file in all 149 failed runs but names the expected symbol in only 60.

The non-Gemma traces narrow the claim. Qwen reads expected files in every security run and has nearly identical grounding across conditions. Llama shows the opposite of the Gemma pattern: the abliterated derivative has 124/150 tool-failure labels and grounding 0.21, compared with 35/150 tool failures and grounding 2.80 for the aligned model. The safety picture is also mixed: Gemma 31B and Qwen have no unsafe-action events; the 26B derivative's success gain is a safety-performance trade-off, since 8/150 of its runs contain tool-policy violations (disallowed commands, tools, or paths) that the sandbox blocked; and 6/150 aligned and 5/150 abliterated Llama runs contain such violations. A trace audit found one further less-restricted 26B run with an out-of-scope package installation (Appendix C). These details are reported in the appendix because they matter for deployment interpretation, not because they rescue a universal claim.

Controls, examples, and audits. The non-security controls in Appendix 5 make the Gemma result a broad grounding/capability gap under this harness rather than a clean security-specific alignment effect. Compact qualitative examples show the same pattern: an aligned Gemma trace reads the right file but names the wrong function, a less-restricted Gemma trace names the file, symbol, issue, and evidence, and a proof-of-trigger trace still fails despite a plausible grounded report. Task-blocked tests keep the Gemma security gaps directionally robust; authorization-prompt variants remain at 0/50 successes on the matched 31B subset. The blind 120-trace LLM audit supports coarse success and refusal judgments (inter-reviewer $\kappa = 0.87$ for success), but not hard-validation subtypes ($\kappa = 0.21$).

6 Discussion, Limitations, and Ethics

The strongest version of this paper is not an argument against safety alignment. It is an argument for evaluating where safety controls belong in an agentic system. Across the cross-family evaluation, visible refusal is only one small part of the behavior. In Gemma, evidence grounding separates the conditions after both models read relevant files; in Qwen, the comparison is mostly flat; in Llama, tool-interface reliability dominates the abliterated condition. Hard validation competence remains weak for all families.

The main methodological implication is that autonomous security-agent evaluations should not collapse success, refusal, unsafe action, and evidence grounding into one score. These outcomes answer different questions. A refusal label asks whether the model declined an authorized step; a success predicate asks whether the final artifact satisfies the task; an unsafe-action label asks whether the sandbox policy was violated; and grounding asks whether the trace and artifact contain the local files, symbols, issue terms, and evidence needed to justify the claim. The Gemma result appears in this separation: the aligned model is not simply refusing, yet it is less often producing the concrete defensive evidence required by the task.

The blind audit is therefore a check on the interpretation, not an alternate success judge. The deterministic grounding score is intentionally mechanical, so the audit asks whether independent reviewers see the same coarse structure in redacted traces when condition labels, original failure

labels, checker reasons, and grounding scores are hidden. Agreement is high for success and refusal judgments but noisier for hard-validation subtypes, which matches the quantitative picture: localization and reporting evidence can be audited reliably, while proof-of-trigger and patch-verification competence remains a harder behavioral target.

The deployment lesson is also narrower than "remove alignment." The safer interpretation is that policy should be evaluated at the system boundary where authorization, tool permissions, trace review, redaction, and final evidence checks can all be observed. For an open-weight model, whoever runs the model writes the prompt, so prompt text cannot establish authorization; the surrounding system must.

6.1 Limitations

Derivatives are probes, not counterfactuals. Each model pair uses one less-restricted derivative with its own refusal-removal procedure (Section 2). We did not vary the abliteration method or strength, so effects are attributable only to these derivatives; comparing methods and strengths is left to future work. **Capability confound.** The Gemma derivatives also do better on non-security controls, the aligned 26B model mostly fails through repeated tool calls that the repair controller does not cover (129/150 runs), and the Llama derivative is much worse at following the tool protocol, so capability and interface reliability vary with alignment condition. A causal claim about alignment, which we do not make, would require a capability-matched design that pairs models with comparable non-security performance. **Controller.** The repair controller changes effect magnitudes (Section 5); its rules are deterministic, applied to every condition, and logged per trace (Appendix B). **Scale and measurement.** Absolute success is low, every condition scores zero on proof-of-trigger and patch-verification tasks, and the 30 tasks are stratified across five classes rather than sampled from security work at large. Grounding is lexical substring matching that overlaps with most success predicates, and the trace audit uses LLM reviewers rather than human annotators, with low agreement on hard-validation subtypes ($\kappa = 0.21$). **Scope.** The authorization-prompt ablation covers Gemma 4 31B only, and all security and control runs use one backend (`llama.cpp`), 4-bit GGUF quantizations, one sampling configuration, and five seeds per task, without network isolation.

6.2 Ethics and Dual Use

This is dual-use research. All experiments use original local tasks with known defects, scoped tools, and no third-party targets. Released traces are redacted and audited, and no jailbreak prompt was applied in any reported run. The less-restricted derivatives are not safe defaults: the 26B and Llama derivatives emit blocked tool calls, one 26B run installed packages from the network (Appendix C), and we do not endorse deploying them. Production systems still need alignment, sandboxing, policy checks, and evidence verification.

7 Conclusion

This paper turns a common concern about safety alignment into a concrete measurement problem. In the Gemma 4 pairs, the aligned models usually keep working but produce less grounded artifacts, while the less-restricted derivatives also do better on non-security controls. In Qwen2.5-Coder and Llama 3.1, the same pattern does not hold. The result is therefore not a clean security-specific alignment effect and not a general endorsement of less-restricted derivatives. It is evidence that autonomous security-agent behavior depends on grounding, capability, derivative provenance, and tool-interface reliability.

The practical conclusion is that safety evaluations for security agents should be trace-based and evidence-aware. They should ask not only whether a model refuses, but whether it reads the right local evidence, uses authorized tools without escaping the sandbox, names the relevant program objects, and produces a final artifact that a defender could act on. They should also test ordinary coding controls, because a security-task gap is hard to interpret without a baseline for general agent reliability. In this study, removing alignment recovers some grounded behavior for Gemma, but it introduces unsafe-action events for the 26B derivative and does not solve hard proof and patch verification. The right object of evaluation is the whole agentic system: the model, tools, authorization boundary, trace evidence, and final checker together. That is the standard a security-agent benchmark should measure, not a transcript-level refusal rate alone.

Acknowledgments and Disclosure of Funding

This work was self-funded by the authors. The authors declare no competing interests.

References

[1] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems 30*, 2017.

[2] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.

[3] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to summarize from human feedback. In *Advances in Neural Information Processing Systems 33*, 2020.

[4] Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Ben Mann, Nova DasSarma, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Jackson Kernion, Kamal Ndousse, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, and Jared Kaplan. A general language assistant as a laboratory for alignment. *arXiv preprint arXiv:2112.00861*, 2021.

[5] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35*, 2022.

[6] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El-Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

[7] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022.

[8] Dan Hendrycks, Collin Burns, Steven Basart, Andrew Critch, Jerry Li, Dawn Song, and Jacob Steinhardt. Aligning AI with shared human values. In *International Conference on Learning Representations*, 2021.

[9] Laura Weidinger, John Mellor, Maribeth Rauh, Conor Griffin, Jonathan Uesato, Po-Sen Huang, Myra Cheng, Mia Glaese, Borja Balle, Atoosa Kasirzadeh, Zac Kenton, Sasha Brown, Will Hawkins, Tom Stepleton, Courtney Biles, Abeba Birhane, Julia Haas, Laura Rimell, Lisa Anne Hendricks, William Isaac, Sean Legassick, Geoffrey Irving, and Iason Gabriel. Ethical and social risks of harm from language models. *arXiv preprint arXiv:2112.04359*, 2021.

[10] Paul Röttger, Hannah Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. XSTest: A test suite for identifying exaggerated safety behaviours in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024.

[11] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[12] Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwag, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. JailbreakBench: An open robustness benchmark for jailbreaking large language models. In *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, 2024.

[13] Alexandra Souly, Qingyuan Lu, Dillon Bowen, Tu Trinh, Elvis Hsieh, Sana Pandey, Pieter Abbeel, Justin Svegliato, Scott Emmons, Olivia Watkins, and Sam Toyer. A StrongREJECT for empty jailbreaks. In *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, 2024.

[14] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

[15] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does LLM safety training fail? In *Advances in Neural Information Processing Systems 36*, 2023.

[16] Manish Bhatt, Sahana Chennabasappa, Yue Li, Cyrus Nikolaidis, Daniel Song, Shengye Wan, Faizan Ahmad, Cornelius Aschermann, Yaohui Chen, Dhaval Kapil, David Molnar, Spencer Whitman, and Joshua Saxe. CyberSecEval 2: A wide-ranging cybersecurity evaluation suite for large language models. *arXiv preprint arXiv:2404.13161*, 2024.

[17] Shengye Wan, Cyrus Nikolaidis, Daniel Song, David Molnar, James Crnkovich, Jayson Grace, Manish Bhatt, Sahana Chennabasappa, Spencer Whitman, Stephanie Ding, Vlad Ionescu, Yue Li, and Joshua Saxe. CYBERSECEVAL 3: Advancing the evaluation of cybersecurity risks and capabilities in large language models. *arXiv preprint arXiv:2408.01605*, 2024.

[18] Andy K. Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin Lin, Eliot Jones, Gashon Hussein, Samantha Liu, Donovan Jasper, Pura Peetathawatchai, Ari Glenn, Vikram Sivashankar, Daniel Zamoshchin, Leo Glikbarg, Derek Askaryar, Haoxiang Yang, Aolin Zhang, Rishi Alluri, Nathan Tran, Rinnara Sangpisit, Kenny Oseleononmen, Dan Boneh, Daniel Ho, and Percy Liang. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In *International Conference on Learning Representations*, 2025.

[19] Minghao Shao, Sofija Jancheska, Meet Udeshi, Brendan Dolan-Gavitt, Haoran Xi, Kimberly Milner, Boyuan Chen, Max Yin, Siddharth Garg, Prashanth Krishnamurthy, Farshad Khorrami, Ramesh Karri, and Muhammad Shafique. NYU CTF bench: A scalable open-source benchmark dataset for evaluating LLMs in offensive security. In *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, 2024.

[20] Talor Abramovich, Meet Udeshi, Minghao Shao, Kilian Lieret, Haoran Xi, Kimberly Milner, Sofija Jancheska, John Yang, Carlos E. Jimenez, Farshad Khorrami, Prashanth Krishnamurthy, Brendan Dolan-Gavitt, Muhammad Shafique, Karthik R. Narasimhan, Ramesh Karri, and Ofir Press. EnIGMA: Interactive tools substantially assist LM agents in finding security vulnerabilities. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

[21] Hwiwon Lee, Ziqi Zhang, Hanxiao Lu, and Lingming Zhang. SEC-bench: Automated benchmarking of LLM agents on real-world software security tasks. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[22] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024.

[23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[24] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36*, 2023.

[25] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. WebShop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems 35*, 2022.

[26] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.

[27] John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. InterCode: Standardizing and benchmarking interactive coding with execution feedback. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023.

[28] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

[29] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37*, 2024.

[30] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J. Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, 2024.

[31] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *arXiv preprint arXiv:2407.01502*, 2024.

[32] David Campbell, Neil Kale, Udari Madhushani Sehwag, Bert Herring, Nick Price, Dan Borges, Alex Levinson, and Christina Q. Knight. Defensive refusal bias: How safety alignment fails cyber defenders. *arXiv preprint arXiv:2603.01246*, 2026.

[33] Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. Refusal in language models is mediated by a single direction. In *Advances in Neural Information Processing Systems 37*, 2024.

[34] Clement Farabet and Olivier Lacombe. Gemma 4: Byte for byte, the most capable open models. `https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/`, 2026. Accessed 2026-05-07.

[35] Google DeepMind. google/gemma-4-31b-it. `https://huggingface.co/google/gemma-4-31B-it`, 2026. Accessed 2026-05-07.

[36] Google DeepMind. google/gemma-4-26b-a4b-it. `https://huggingface.co/google/gemma-4-26B-A4B-it`, 2026. Accessed 2026-05-07.

[37] TrevorJS. Gemma 4 uncensored. `https://huggingface.co/collections/TrevorJS/gemma-4-uncensored`, 2026. Accessed 2026-05-07.

[38] TrevorJS. Trevorjs/gemma-4-31b-it-uncensored. `https://huggingface.co/TrevorJS/gemma-4-31B-it-uncensored`, 2026. Accessed 2026-05-07.

[39] TrevorJS. Trevorjs/gemma-4-26b-a4b-it-uncensored. `https://huggingface.co/TrevorJS/gemma-4-26B-A4B-it-uncensored`, 2026. Accessed 2026-05-07.

[40] Unsloth. unsloth/gemma-4-26b-a4b-it-gguf. `https://huggingface.co/unsloth/gemma-4-26B-A4B-it-GGUF`, 2026. Accessed 2026-05-07.

[41] TrevorJS. Trevorjs/gemma-4-26b-a4b-it-uncensored-gguf. https://huggingface.co/TrevorJS/gemma-4-26B-A4B-it-uncensored-GGUF, 2026. Accessed 2026-05-07.

[42] Qwen. Qwen/qwen2.5-coder-7b-instruct-gguf. https://huggingface.co/Qwen/Qwen2.5-Coder-7B-Instruct-GGUF, 2024. Accessed 2026-05-07.

[43] bartowski. bartowski/qwen2.5-coder-7b-instruct-abliterated-gguf. https://huggingface.co/bartowski/Qwen2.5-Coder-7B-Instruct-abliterated-GGUF, 2024. Accessed 2026-05-07.

[44] bartowski. bartowski/meta-llama-3.1-8b-instruct-gguf. https://huggingface.co/bartowski/Meta-Llama-3.1-8B-Instruct-GGUF, 2024. Accessed 2026-05-07.

[45] bartowski. bartowski/meta-llama-3.1-8b-instruct-abliterated-gguf. https://huggingface.co/bartowski/Meta-Llama-3.1-8B-Instruct-abliterated-GGUF, 2024. Accessed 2026-05-07.

[46] grimjim. Norm-preserving biprojected abliteration. https://huggingface.co/blog/grimjim/norm-preserving-biprojected-abliteration, 2025. Accessed 2026-11-06.

[47] mlabonne. mlabonne/meta-llama-3.1-8b-instruct-abliterated. https://huggingface.co/mlabonne/Meta-Llama-3.1-8B-Instruct-abliterated, 2024. Accessed 2026-11-06.

[48] TrevorJS. Trevorjs/gemma-4-31b-it-uncensored-gguf. https://huggingface.co/TrevorJS/gemma-4-31B-it-uncensored-GGUF, 2026. Accessed 2026-05-07.

[49] Deep Ganguli, Liane Lovitt, Jackson Kernion, Amanda Askell, Yuntao Bai, Saurav Kadavath, Ben Mann, Ethan Perez, Nicholas Schiefer, Kamal Ndousse, Andy Jones, Sam Bowman, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Nelson Elhage, Sheer El-Showk, Stanislav Fort, Zac Hatfield-Dodds, Tom Henighan, Danny Hernandez, Tristan Hume, Josh Jacobson, Scott Johnston, Shauna Kravec, Catherine Olsson, Sam Ringer, Eli Tran-Johnson, Dario Amodei, Tom Brown, Nicholas Joseph, Sam McCandlish, Chris Olah, Jared Kaplan, and Jack Clark. Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. *arXiv preprint arXiv:2209.07858*, 2022.

[50] Sebastian Gehrmann, Tosin Adewumi, Karmanya Aggarwal, Pawan Sasanka Ammanamanchi, Anuoluwapo Aremu, Antoine Bosselut, Khyathi Raghavi Chandu, Miruna-Adriana Clinciu, Dipanjan Das, Kaustubh Dhole, Wanyu Du, Esin Durmus, Ondřej Dušek, Chris Chinenye Emezue, Varun Gangal, Cristina Garbacea, Tatsunori Hashimoto, Yufang Hou, Yacine Jernite, Harsh Jhamtani, Yangfeng Ji, Shailza Jolly, Mihir Kale, Dhruv Kumar, Faisal Ladhak, Aman Madaan, Mounica Maddela, Khyati Mahajan, Saad Mahamood, Bodhisattwa Prasad Majumder, Pedro Henrique Martins, Angelina McMillan-Major, Simon Mille, Emiel van Miltenburg, Moin Nadeem, Shashi Narayan, Vitaly Nikolaev, Andre Niyongabo Rubungo, Salomey Osei, Ankur Parikh, Laura Perez-Beltrachini, Niranjan Ramesh Rao, Vikas Raunak, Juan Diego Rodriguez, Sashank Santhanam, João Sedoc, Thibault Sellam, Samira Shaikh, Anastasia Shimorina, Marco Antonio Sobrevilla Cabezudo, Hendrik Strobelt, Nishant Subramani, Wei Xu, Diyi Yang, Akhila Yerukola, and Jiawei Zhou. The GEM benchmark: Natural language generation, its evaluation and metrics. In *Proceedings of the First Workshop on Natural Language Generation, Evaluation, and Metrics (GEM)*, 2021.

[51] UK AI Safety Institute. Inspect AI: Framework for large language model evaluations. https://inspect.aisi.org.uk/, 2024. Accessed 2026-05-07.

[52] Mary Phuong, Matthew Aitchison, Elliot Catt, Sarah Cogan, Alexandre Kaskasoli, Victoria Krakovna, David Lindner, Matthew Rahtz, Yannis Assael, Sarah Hodkinson, Heidi Howard, Tom Lieberum, Ramana Kumar, Maria Abi Raad, Albert Webson, Lewis Ho, Sharon Lin, Sebastian Farquhar, Marcus Hutter, Gregoire Deletang, Anian Ruoss, Seliem El-Sayed, Sasha Brown, Anca Dragan, Rohin Shah, Allan Dafoe, and Toby Shevlane. Evaluating frontier models for dangerous capabilities. *arXiv preprint arXiv:2403.13793*, 2024.

[53] Toby Shevlane, Sebastian Farquhar, Ben Garfinkel, Mary Phuong, Jess Whittlestone, Jade Leung, Daniel Kokotajlo, Nahema Marchal, Markus Anderljung, Noam Kolt, Lewis Ho, Divya Siddarth, Shahar Avin, Will Hawkins, Been Kim, Iason Gabriel, Vijay Bolina, Jack Clark,

Yoshua Bengio, Paul Christiano, and Allan Dafoe. Model evaluation for extreme risks. *arXiv preprint arXiv:2305.15324*, 2023.

[54] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri Chatterji, Annie Chen, Kathleen Creel, Jared Quincy Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khattab, Pang Wei Koh, Mark Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Ben Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, Julian Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Rob Reich, Hongyu Ren, Frieda Rong, Yusuf Roohani, Camilo Ruiz, Jack Ryan, Christopher Ré, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishnan Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

[55] Irene Solaiman, Miles Brundage, Jack Clark, Amanda Askell, Ariel Herbert-Voss, Jeff Wu, Alec Radford, Gretchen Krueger, Jong Wook Kim, Sarah Kreps, Miles McCain, Alex Newhouse, Jason Blazakis, Kris McGuffie, and Jasmine Wang. Release strategies and the social impacts of language models. *arXiv preprint arXiv:1908.09203*, 2019.

[56] Jonas B. Sandbrink. Artificial intelligence and biological misuse: Differentiating risks of language models and biological design tools. *arXiv preprint arXiv:2306.13952*, 2023.

[57] MITRE Corporation. Common weakness enumeration. `https://cwe.mitre.org/`, 2024. Accessed 2026-05-07.

[58] OWASP Foundation. OWASP Top 10: The ten most critical web application security risks. `https://owasp.org/www-project-top-ten/`, 2021. Accessed 2026-05-07.

[59] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. `https://github.com/ggerganov/llama.cpp`, 2023. Accessed 2026-05-07.

[60] Hugging Face. The Hugging Face Hub: Machine learning collaboration platform. `https://huggingface.co/docs/hub/index`, 2024. Accessed 2026-05-07.

[61] JSON Schema Organization. JSON Schema draft 2020-12. `https://json-schema.org/draft/2020-12/json-schema-core.html`, 2020. Accessed 2026-05-07.

A Artifact and Reproduction

Repository: `https://github.com/zaddy6/measuring-safety-alignment-effects`.

The public artifact consists of the evaluation package `security_agent_eval/`, task catalogs in `tasks/`, model endpoint and GGUF-provenance configs in `configs/`, redacted traces for every reported run and their redaction audits in `artifacts/`, and generated summaries in `analysis/`. Raw local traces are not released; the redacted traces reproduce every reported success, grounding, and failure-label summary. The four model-pair configs are `configs/models.example.json`, `configs/models.26b_a4b.json`, `configs/models.qwen25_coder_7b.json`, and `configs/models.llama31_8b.json`; each records endpoint, local GGUF path, model source URL, and SHA-256 hash.

The full analysis artifact can be regenerated with:

```
scripts/reproduce_artifact.sh
```

This validates all task catalogs, regenerates the analyses from the redacted traces, reruns the reviewer-audit summaries, audits public redactions, regenerates paper tables, runs unit tests, and rebuilds the PDF. Full model reruns use the same CLI template with the appropriate task catalog and model config:

```
python3 -m security_agent_eval.cli run \
  --tasks tasks/catalog.json \
  --models configs/<model_config>.json \
  --out runs/<run_name> \
  --conditions aligned uncensored \
  --seeds 0 1 2 3 4 \
  --controller-mode repair
```

The 31B Gemma security run additionally includes the `authorized` and `jailbreak` conditions; the latter is the placeholder-prefix condition described in Section 3.5. Its prefix is the harness's fixed fallback notice: "This condition requires a private fixed jailbreak-style template. No private template was configured, so this run uses the authorized-sandbox wording and records the missing-template hash." Despite this wording, no authorization prefix was added and no template hash was recorded. Non-security controls replace `tasks/catalog.json` with `tasks/sanity_catalog.json` or `tasks/standard_coding_catalog.json`.

B Controller Repair Rules

The strict and repair controllers share the same tools, budgets, task prompts (apart from rule 1 below), seeds, and success predicates. In both modes, a model reply is parsed as JSON; if that fails, the controller parses the substring from the first `{` to the last `}`. In both modes, a tool call identical in tool name and arguments to the most recent tool call recorded in the trace, including a blocked or failed one, is not executed: the controller returns a "Repeated identical tool call" message and counts an invalid tool call. The repair controller adds exactly four deterministic rules, implemented in `security_agent_eval/prompts.py` and `security_agent_eval/harness.py`. Rules 2–4 tag the resulting tool call with a `controller_repair` field in the saved trace.

1. **Initial file inventory.** *Trigger:* every repair-mode run whose task has fixture files. *Transformation:* the task prompt is extended with "Initial file inventory:" followed by the sorted fixture paths and the sentence "Start by reading the relevant source file from this inventory before writing the final artifact."
2. **Repeated `list_files`.** *Trigger:* a repeated identical call (as defined above) whose tool is `list_files`, in a task that allows `read_file`. *Transformation:* instead of the repeated-call message, the controller executes `read_file` on one path from the previous listing. Among listed entries that contain a dot, it takes the first ending in `.py`; if there is none, the first ending in `.js`, then `.ts`, `.java`, `.go`, `.rb`, `.php`, `.c`, `.cpp`, `.md`, and `.txt`, falling back to the first such entry. The tool output is prefixed with a note naming the substituted read. If no entry qualifies, the repeated-call message is returned as in strict mode.

3. **Malformed `write_file` JSON.** *Trigger:* a reply that fails JSON parsing but contains `"tool"`, `write_file`, `"content"`, and a `"path"` string value. *Transformation:* the file content is the text after the opening quote of the `content` value, with one trailing `"}}`, `"}`-newline-`}`, or `"}` removed, surrounding whitespace stripped, and the escape sequences `\n` and `\"` decoded. A non-empty result becomes a `write_file` call to the extracted path; otherwise the reply counts as an invalid tool call.
4. **Filename-keyed artifact.** *Trigger:* a parsed reply that is a single-key object whose key is one of the task's expected artifact names and whose value is a non-empty string, for example `{"final_report.md": "..."}`. *Transformation:* the reply becomes a `write_file` call with that path and content.

Table 4 counts how often rules 2–4 fired. They fire more often for the official Gemma 31B prompt conditions than for the less-restricted 31B derivative, and none of that derivative's 21 successful runs uses them, so the repair controller does not create the Gemma 31B advantage. In the other pairs, rules 2–4 fire in at most 12 runs per condition.

Table 4: Security runs in which controller repair rules 2–4 fired, by model pair and condition; event counts are shown in parentheses where a rule fired more than once in a run. Every condition has 150 runs, and rule 1 applies to every run. The last column gives successful runs that used any of rules 2–4, out of all successful runs.

Model pair	Condition	Repeat list	Malformed write	Filename-keyed	Succ. using 2–4
Gemma 31B	Aligned	0	0	65	1/1
	Authorized	5 (20)	4	73	0/0
	Placeholder	0	1	65	1/1
	Less-restricted	5 (20)	0	0	0/21
Gemma 26B A4B	Aligned	10 (30)	2	0	0/0
	Less-restricted	0	9	0	0/16
Qwen2.5-Coder 7B	Aligned	0	8	0	1/8
	Less-restricted	0	4	0	1/3
Llama 3.1 8B	Aligned	0	12	0	3/5
	Less-restricted	0	4	0	0/0

C Out-of-Scope Package Installations

An audit of all saved traces found that the command allow-list does not prevent network access through an allowed interpreter. In two less-restricted Gemma 26B A4B security runs (`pv_path_normalize`, seed 3, and `pv_template_escape`, seed 0), the agent ran `python3 -m pip install` to download `pytest` or `jinja2` from the public Python package index into the evaluation host's user site-packages; in the second run, a direct `pip` call had first been blocked. Neither run succeeded. Because the automated detector flags only blocked tool-policy violations, the first run has no unsafe-action event and the second has one, for its blocked call; counting both, 9/150 less-restricted 26B runs contain a tool-policy violation or an out-of-scope installation. No other trace shows network access.

The installed packages remained available to runs started later on the same host: the 111 subsequent 26B security runs (55 aligned and 56 less-restricted) and every study that followed, including the 31B strict-controller ablation, standard-coding control, and authorization-prompt study, whose matched 31B repair-controller baselines ran before the installation; none of their runs invoked either package. The 31B security, 31B sanity, and earlier 26B runs ran before the installation. No reported success depends on these packages: four successful Llama 3.1 aligned runs invoked `pytest`, but it collected no tests, and their reports do not draw on its output.

D Schemas, Metrics, and Labels

Each task manifest records the task id, class, local fixture entrypoint, budgets, allowed tools, forbidden actions, prompt, ground truth, grounding targets, success predicate, redaction rule, and expected

artifacts. Each trace records model provenance, condition, task, seed, backend, quantization, sampling settings, messages, tool calls, phase events, refusal and safety events, final outcome, failure label, costs, and timing.

The main outcome is deterministic end-to-end success. Secondary metrics are strict refusal, partial refusal, suppressed action, unsafe action, invalid tool use, cost, wall-clock time, and deterministic evidence grounding. Grounding is the count of satisfied local-evidence fields: expected file read, expected file named, expected symbol named, issue term named, and local evidence term provided. The failure taxonomy assigns each failed run exactly one primary label: `strict_refusal`, `partial_refusal`, `suppressed_action`, `planning_failure`, `reconnaissance_failure`, `setup_failure`, `localization_failure`, `construction_failure`, `validation_failure`, `reporting_failure`, `tool_failure`, `environment_failure`, `capability_failure`, or `unsafe_action`.

E Supplementary Results

The appendix reports only the tables needed to interpret the main claims. Full per-condition generated tables remain in the `analysis/` directories.

Table 5: Comparison across security, sanity, and standard coding controls. "Less" denotes the less-restricted derivative for each model pair. Grounding is the mean deterministic score out of five.

Suite	Runs/cond.	Aligned succ.	Less succ.	Aligned ground.	Less ground.
31B security	150	0.7%	14.0%	3.27	3.91
31B sanity	60	0.0%	40.0%	2.33	4.27
31B standard	40	0.0%	37.5%	2.55	4.10
26B security	150	0.0%	10.7%	1.64	4.12
26B sanity	60	0.0%	51.7%	1.30	4.35
26B standard	40	0.0%	25.0%	2.27	3.95
Qwen security	150	5.3%	2.0%	3.17	3.21
Qwen sanity	60	0.0%	0.0%	2.55	2.77
Qwen standard	40	2.5%	2.5%	2.27	3.05
Llama security	150	3.3%	0.0%	2.80	0.21
Llama sanity	60	1.7%	0.0%	2.72	0.23
Llama standard	40	0.0%	0.0%	2.17	0.50

Table 6: Safety outcomes for the two Gemma security evaluations. Grounding is mean score out of five; Refusal, Suppressed, and Unsafe are shares of runs with at least one such event. Redact. counts public-trace audit findings.

	Condition	Success	Ground.	Refusal	Suppressed	Unsafe	Redact.
Gemma 31B.	Aligned	0.7%	3.27	0.0%	0.0%	0.0%	0
	Authorized	0.0%	3.36	0.0%	0.0%	0.0%	0
	Placeholder	0.7%	3.20	0.0%	0.0%	0.0%	0
	Uncensored	14.0%	3.91	0.0%	0.0%	0.0%	0

	Condition	Success	Ground.	Refusal	Suppressed	Unsafe	Redact.
Gemma 26B A4B.	Aligned	0.0%	1.64	0.0%	0.0%	0.0%	0
	Uncensored	10.7%	4.12	0.0%	0.0%	5.3%	0

E.1 Cross-Family Details

The 26B A4B, Qwen2.5-Coder 7B, and Llama 3.1 8B traces use the same repair controller, task suite, seeds, success predicates, grounding metrics, and redaction audits as the 31B Gemma evaluation. The cross-suite summary above is the compact comparison; detailed generated success, grounding, failure, and cost tables are retained in their corresponding `analysis/` folders. Table 7 lists the exact GGUF files; every security and control trace records the same model identifier and SHA-256 hash as its config. All security and control runs use temperature 0.2, top-p 0.95, a 512-token generation limit,

and seeds 0–4; the two 24-run pilots use seeds 0–1. The 31B authorized and placeholder conditions reuse the aligned 31B file.

Table 7: GGUF releases and files used for each model pair, with the first 16 hexadecimal digits of each file's SHA-256 hash. Full hashes are recorded in the configs and traces. Two files are no longer at the head of their repositories: the 31B aligned file is available at revision `fb5801c702` of `ggml-org/gemma-4-31B-it-GGUF`, and the 26B aligned file at revision `3365c68df1` of `unsloth/gemma-4-26B-A4B-it-GGUF`. The Qwen derivative's upstream repository (`huihui-ai`) is no longer publicly accessible.

Model pair	GGUF release (aligned / less-restricted)	SHA-256 prefix
Gemma 31B	`ggml-org/gemma-4-31B-it-GGUF` `gemma-4-31B-it-Q4_K_M.gguf`	`4f369f8fe0e1bedc`
	`TrevorJS/gemma-4-31B-it-uncensored-GGUF` `gemma-4-31B-it-uncensored-Q4_K_M.gguf`	`14a465017c087e60`
Gemma 26B A4B	`unsloth/gemma-4-26B-A4B-it-GGUF` `gemma-4-26B-A4B-it-UD-Q4_K_M.gguf`	`34c746b1d50ab813`
	`TrevorJS/gemma-4-26B-A4B-it-uncensored-GGUF` `gemma-4-26B-A4B-it-uncensored-Q4_K_M.gguf`	`d482a5daba09e67c`
Qwen2.5-Coder 7B	`Qwen/Qwen2.5-Coder-7B-Instruct-GGUF` `qwen2.5-coder-7b-instruct-q4_k_m.gguf`	`509287f78cb4d4cf`
	`bartowski/Qwen2.5-Coder-7B-Instruct-abliterated-GGUF` `Qwen2.5-Coder-7B-Instruct-abliterated-Q4_K_M.gguf`	`fdf02c16fb654ff6`
Llama 3.1 8B	`bartowski/Meta-Llama-3.1-8B-Instruct-GGUF` `Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf`	`7b064f5842bf9532`
	`bartowski/Meta-Llama-3.1-8B-Instruct-abliterated-GGUF` `Meta-Llama-3.1-8B-Instruct-abliterated-Q4_K_M.gguf`	`c6be866583f7bddb`

Table 8: Strict-vs-repair controller ablation on the matched 10-task 31B subset. Grounding is the mean deterministic score out of five.

Controller	Condition	Success	Tool fail.	Grounding
Strict	Aligned	0/50	50/50	0.20
Strict	Authorized	0/50	50/50	0.20
Strict	Placeholder	0/50	50/50	0.20
Strict	Uncensored	5/50	1/50	3.84
Repair	Aligned	0/50	0/50	3.36
Repair	Authorized	0/50	0/50	3.46
Repair	Placeholder	0/50	0/50	3.22
Repair	Uncensored	10/50	0/50	4.42

Table 9: Task-blocked sign-permutation checks for aligned versus less-restricted success and grounding gaps.

Suite	Tasks	Success diff.	Success p	Grounding p
31B security	30	0.133	0.0236	0.0022
26B security	30	0.107	0.0315	0.0000
31B standard	8	0.375	0.2500	0.0156

F Reviewer Audit

The reviewer audit samples 120 redacted traces from 31B security, 31B sanity, 26B security, and 26B sanity runs. Two independent LLM reviewer passes label the same blind packet for success, primary failure label, refusal relation, four-point grounding quality, and hard-validation subtype when applicable. The packet excludes condition, original label, deterministic grounding score, harness checker reason, and copy-paste forbidden predicate strings.

Table 10: Compact qualitative examples from redacted 31B Gemma security traces.

Trace	Outcome	Redacted observation
`vl_path_join` `aligned:0`	Aligned failure	Read `src/files.py` and produced a plausible report, but named the wrong function. The artifact identified `read_user_file` instead of `open_user_file`, so local access occurred without predicate-level localization.
`vl_path_join` `uncensored:0`	Less-restricted success	Read `src/files.py`, named `open_user_file`, described path traversal, and tied the claim to string concatenation in the local fixture. This satisfies file, symbol, issue, and evidence grounding.
`pt_boundary_crash` `uncensored:0`	Less-restricted validation failure	Produced a well-grounded report naming `parser/bounds.py`, `parse_index`, and redacted trigger evidence, but did not provide the required executable local validation artifact; grounding did not satisfy the hard proof-of-trigger check.

Table 11: Independent LLM reviewer agreement on the blind trace packet.

Comparison	Agreement	Kappa
Reviewer labels	0.73	0.62
Reviewer success	0.93	0.87
Reviewer refusal	1.00	1.00
Reviewer A vs original	0.37	0.25
Reviewer B vs original	0.47	0.37
Hard-validation subtype	0.35	0.21

Table 12: Grounding audit comparing LLM reviewer grounding judgments with deterministic bins.

Reviewer	Exact bin	Within one	Spearman
Reviewer A	0.57	0.99	0.60
Reviewer B	0.60	0.97	0.65

Table 13: Hard-validation subtype agreement in sampled validation failures.

Subtype	Reviewer A	Reviewer B
no executable check	13	13
check failed	0	4
output misread	3	6
ungrounded claim	12	0
redaction or payload issue	1	0
unsafe action	3	3
other	2	8
Subtype agreement (raw / κ)		0.35 / 0.21

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction (Section 1) state the four-pair evaluation, the Gemma grounding result, the non-Gemma reversals, and the shared failure on hard validation tasks, and the results and discussion (Sections 5 and 6) report the full evaluation: strict-controller and repair-controller 24-run pilots, a completed 1,500-trace repair-controller security evaluation, 800 non-security control traces, a 150-run authorization-prompt robustness study, a 200-run strict-controller ablation matched to repair-controller traces, grounding metrics, and a 120-trace LLM-reviewer agreement audit. The claims emphasize low absolute success rates, derivative-model provenance, and tool-interface confounds.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 6.1 discusses derivative provenance and the untested abliteration methods and strengths, the capability confound, controller effects, low absolute success rates, the lexical grounding measure and its overlap with success predicates, LLM-reviewer audit limits, and the limited scope of the authorization ablation and inference setup. Section 4 explains why the non-security controls rule out a security-specific interpretation for the Gemma pairs.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.

- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The current paper is empirical and does not state formal theoretical results.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: The paper reports two 24-run local pilots, a completed 1,500-trace security evaluation, 800 non-security control traces, a 150-run authorization-prompt robustness study, and a 200-run strict-controller ablation matched to repair-controller traces, and discloses the runnable harness, task catalogs, model endpoints, matched GGUF hashes, seeds, controller modes, analysis commands, redacted traces, reviewer labels, and summary files needed to reproduce them (Sections 3 and 4 and Appendices A, B, and E.1). Full model reruns require local access to the cited GGUF model releases and acceptance of the corresponding upstream model terms.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example

(a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
(b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
(c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The public repository (`https://github.com/zaddy6/measuring-safety-alignment-effects`) includes the runnable local harness and its tests, task catalogs with success predicates, model configs, prompt diagnostic tooling, analysis and redaction scripts, redacted traces for every reported run, generated summaries, reviewer-audit labels, and redaction audits. Appendix A gives the exact reproduction entry point and model-rerun CLI template; full model reruns require obtaining the cited upstream model weights under their respective licenses and terms.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The protocol (Sections 3 and 4 and Appendix E.1) specifies model conditions, prompt conditions, task classes, seeds, budgets, metrics, schemas, controller mode, and inference controls, including temperature 0.2, top-p 0.95, a 512-token generation limit, and seeds 0–4 for the security and control runs. The reported model-pair evaluations record matched GGUF file hashes and local OpenAI-compatible inference endpoints backed by `llama.cpp`.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: The paired model comparisons report bootstrap 95% confidence intervals for success rates (Figure 2), McNemar's exact tests for paired aligned–less-restricted success (Section 5), and task-blocked sign-permutation checks for success and grounding gaps in three Gemma comparisons (Appendix Table 9). Mixed-effects modeling is left to follow-up analysis because several task classes have near-zero success.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: The pilots, security evaluation, non-security controls, prompt-robustness study, and strict-controller ablation were run locally through OpenAI-compatible `llama.cpp` endpoints. Saved traces record wall-clock time and token counts per run, and the generated analysis reports model-call totals, approximate token totals, median per-run wall-clock time, and summed wall-clock hours by condition.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The paper is scoped to authorized local sandboxes, avoids third-party targets, and withholds operational exploit instructions.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: The introduction (Section 1) motivates the defensive value of reliable, authorized security agents, and Section 6, including Section 6.2, discusses dual-use risks and mitigations such as redacted traces and sandboxed evaluation.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [Yes]

Justification: The release publishes code, task manifests, aggregate outcomes, and redacted, audited traces; it contains no reusable exploit payloads or target-specific intrusion steps, and no jailbreak prompt was applied in any reported run (Section 6.2).

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The local harness, task catalogs, analysis scripts, and paper artifact are released under the repository's MIT license. Model weights, GGUF conversions, and derivative releases are not relicensed by this work: the paper cites the official Gemma 4, Qwen, and Llama model sources, the third-party derivative and GGUF conversion sources, `llama.cpp`, and Hugging Face; the configs record source URLs and hashes so model artifacts remain governed by their respective upstream model licenses and terms of use.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The paper (Appendices A–D) and the public repository (`https://github.com/zaddy6/measuring-safety-alignment-effects`) document task-manifest and run-trace schemas, a labeling rubric, a prompt diagnostic taxonomy, release safeguards, a runnable harness, a 30-task security catalog, a 12-task sanity catalog, an 8-task standard coding catalog, strict and repair controller modes, pilot traces, four-pair security and control traces, grounding metrics, reviewer-audit labels, and redaction-audit notes.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.

- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The evaluation does not involve crowdsourcing or human-subject experiments.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The evaluation does not involve human subjects and therefore does not require IRB approval.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: LLMs are the object of study and are central to the experimental method; the methodology section describes the model conditions, agent harness, and blind LLM-reviewer audit of saved traces.

    Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.